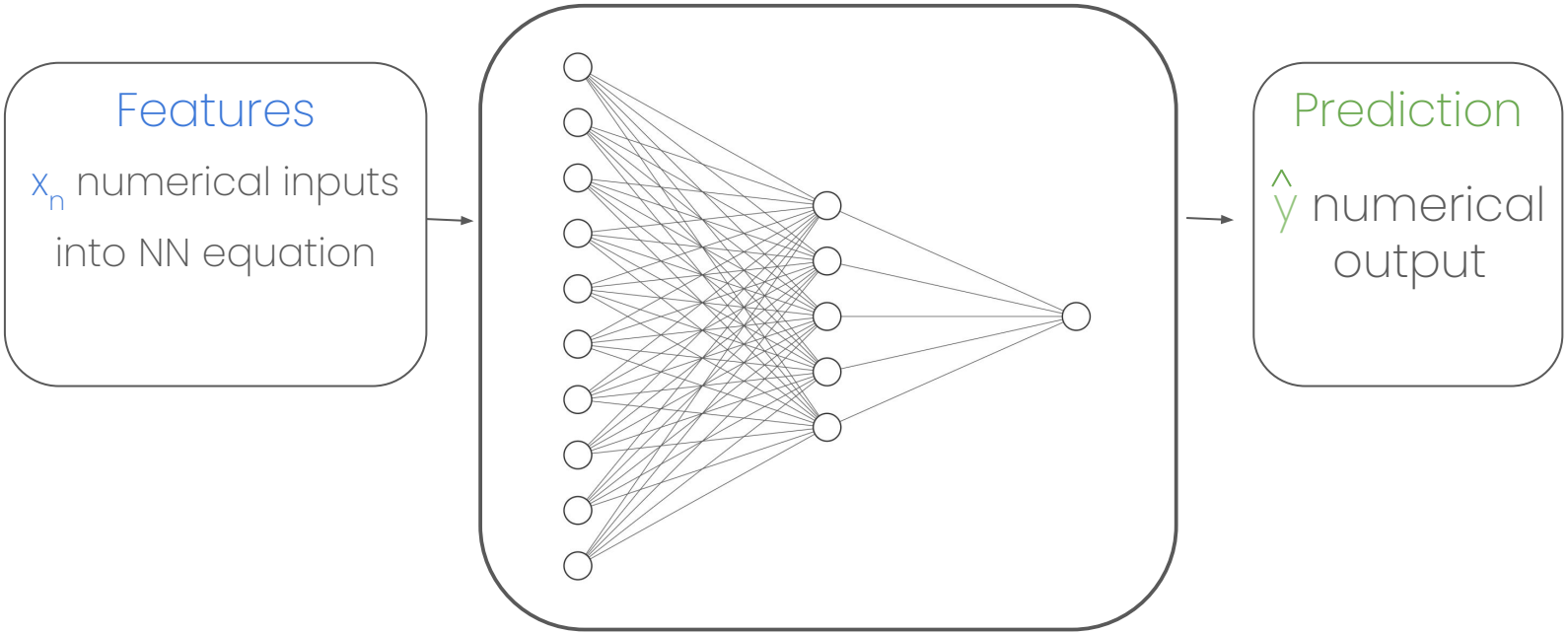
Computer Vision

Recall Neural Networks



How would NNs process images?

Images

32px

32px

every pixel = (r, g, b)

Flatten into 32 x 32 x 3 = 3072

input vector

Features

x_0

x_1

...

...

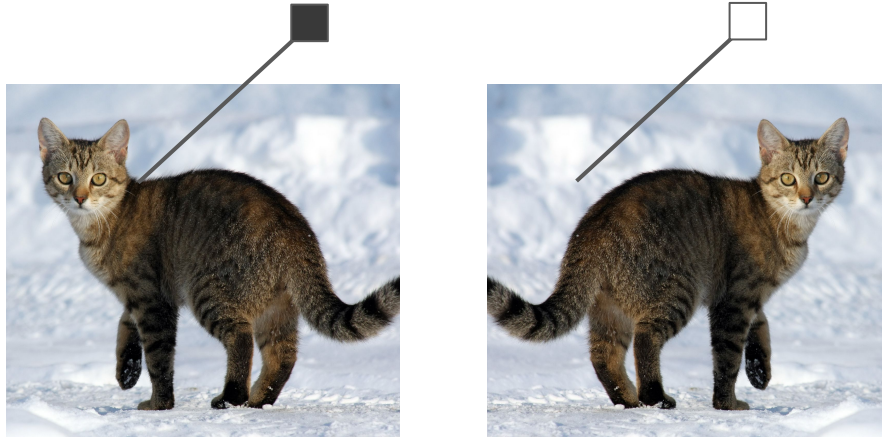
x_{3070}

x_{3071}

But in images, spatial information matters!

Problem with flattening:

one input neuron sees two colors for the same label



Convolutional Neural Networks

Looks for features instead of individual pixels

CNN: Convolution Operation

Kernels / Filters (ex: 3x3):

a_1	a_2	a_3
a_4	a_5	a_6
a_7	a_8	a_9

Dot product:

x_1	x_2	x_3
x_4	x_5	x_6
x_7	x_8	x_9

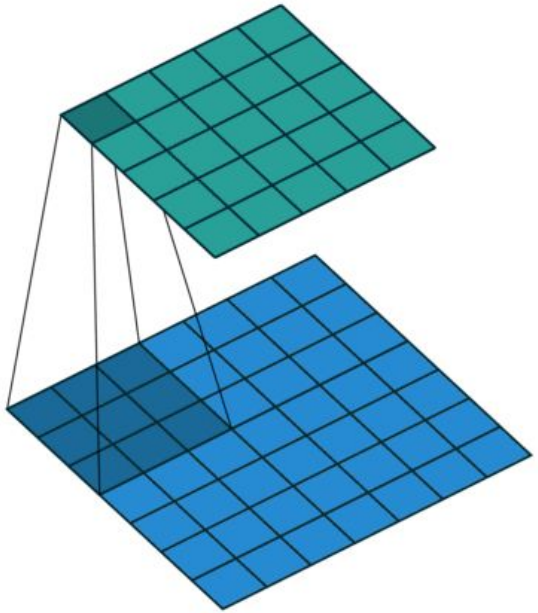
$\bullet$

y_1	y_2	y_3
y_4	y_5	y_6
y_7	y_8	y_9

$$= \sum_{i=1}^{9} x_i y_i$$

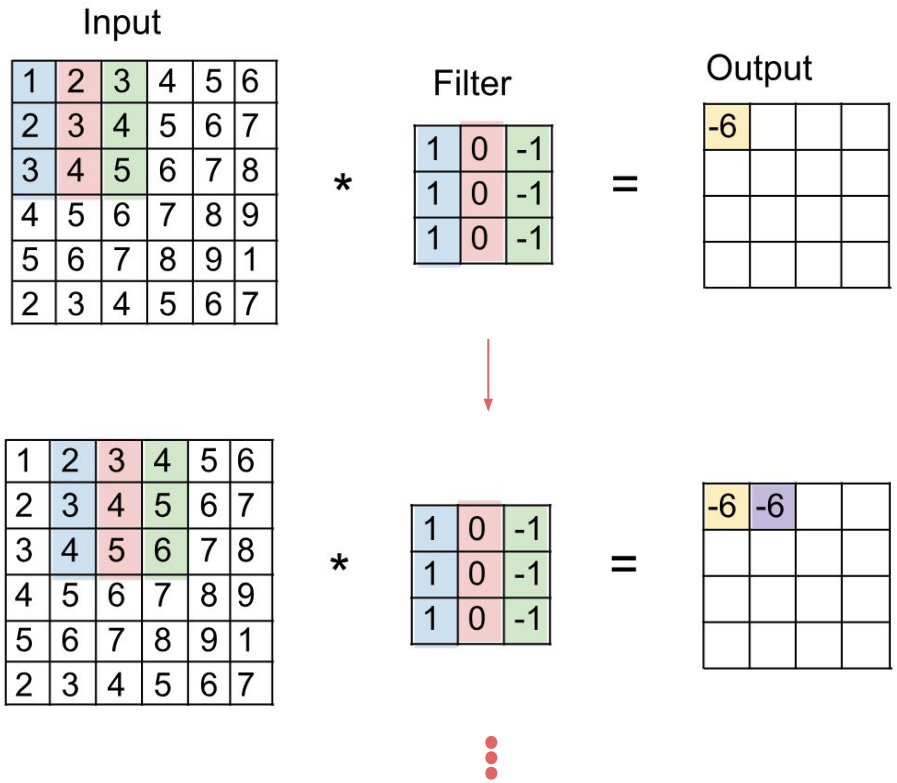
CNN: Convolution Operation

Kernels applied over image to produce output matrix



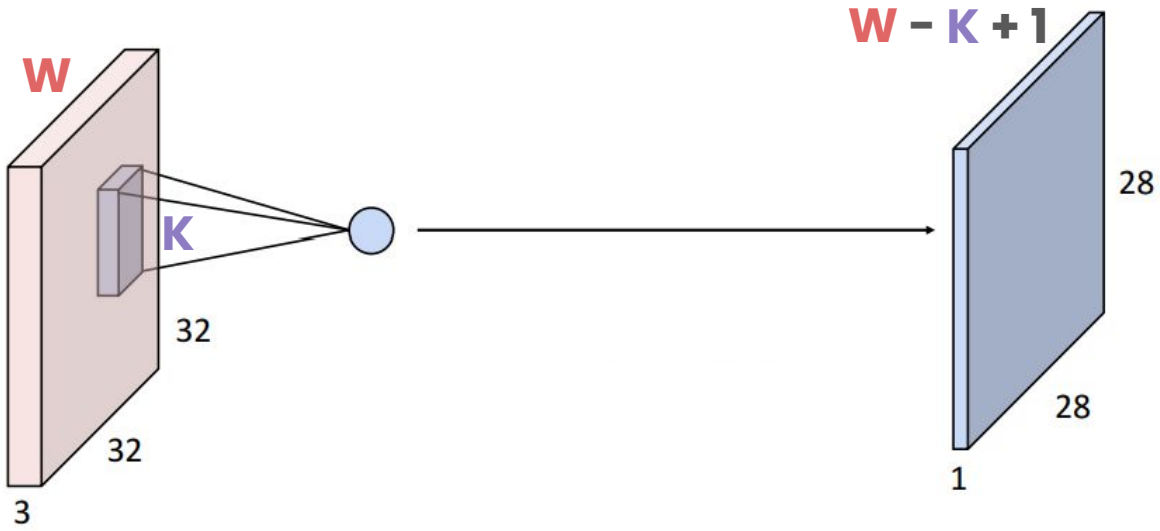
https://medium.com/data-science/computer-vision-convolution-basics-2d0ae3b79346

CNN: Convolution Operation



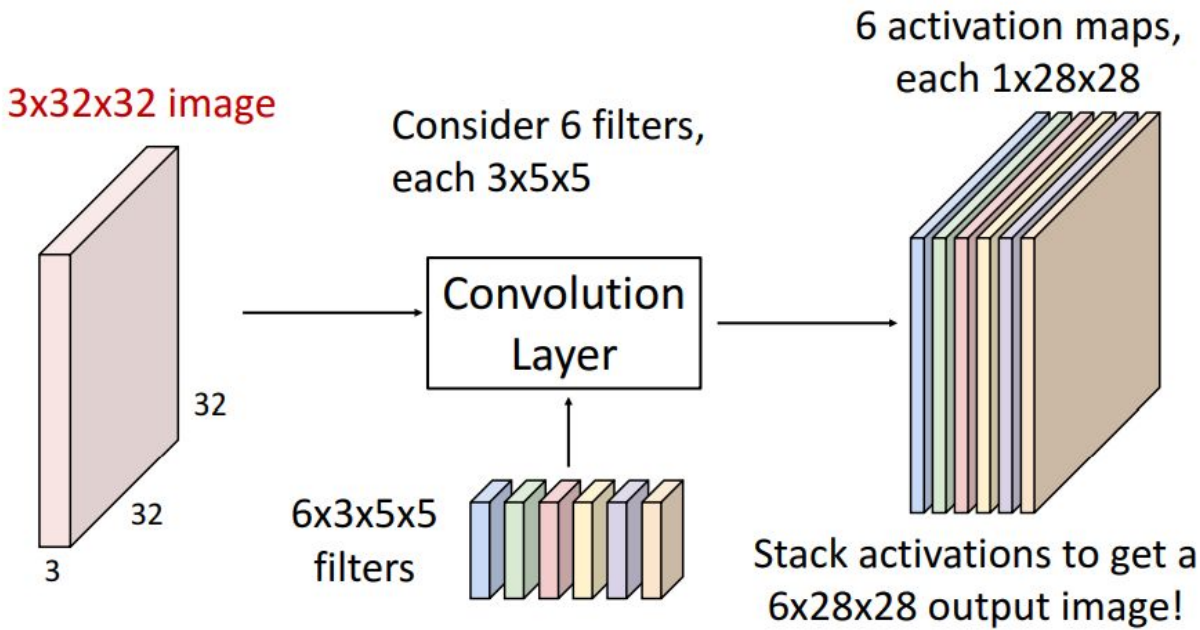
CNN: Convolution Operation

Recall that rgb images are 3 dimensional



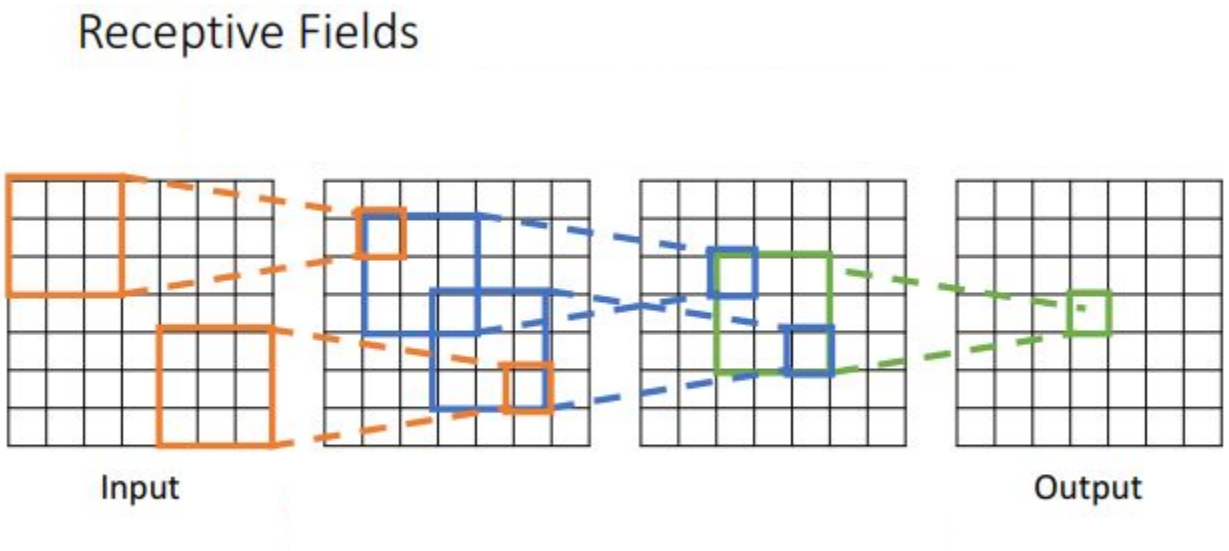
CNN: Convolution Operation

Multiple kernels in one layer



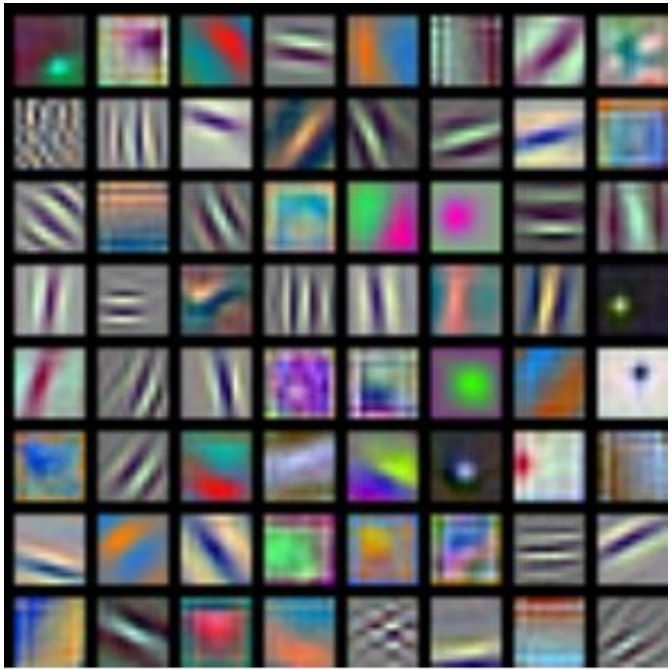
CNN: Convolution Operation

Multiple convolutional layers:



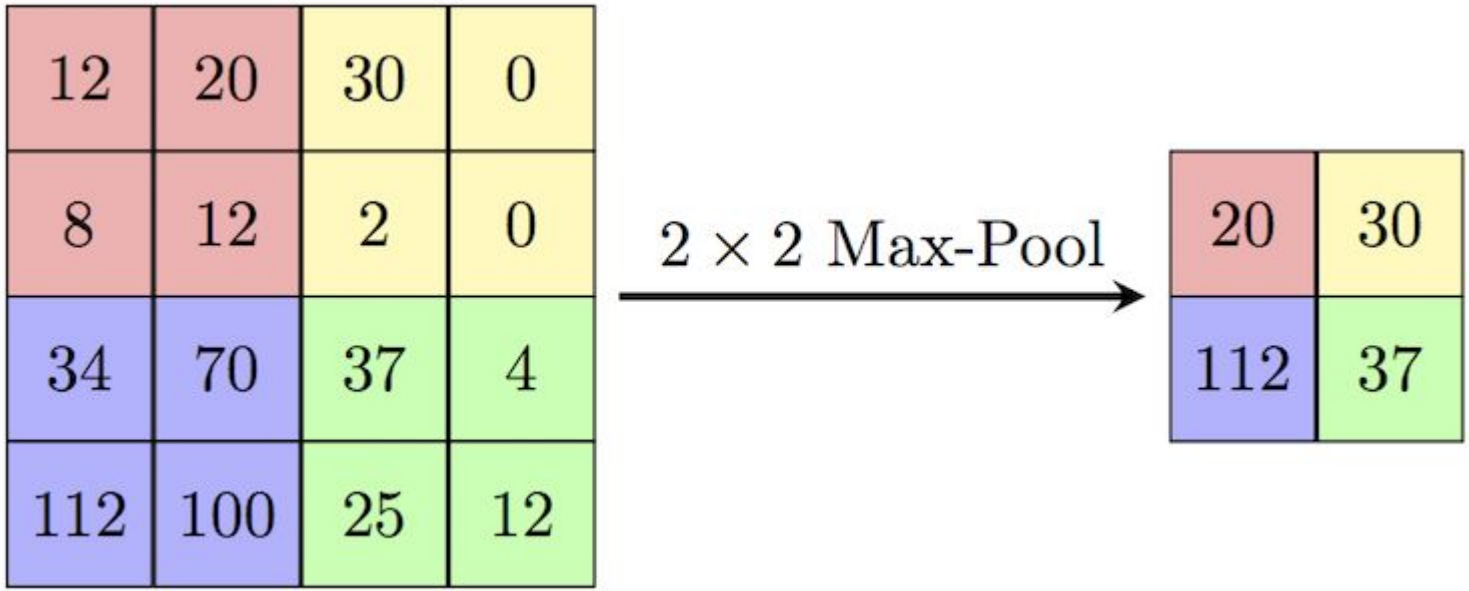
CNN: Convolution Operation

Real life kernels:



CNN: Pooling Layer

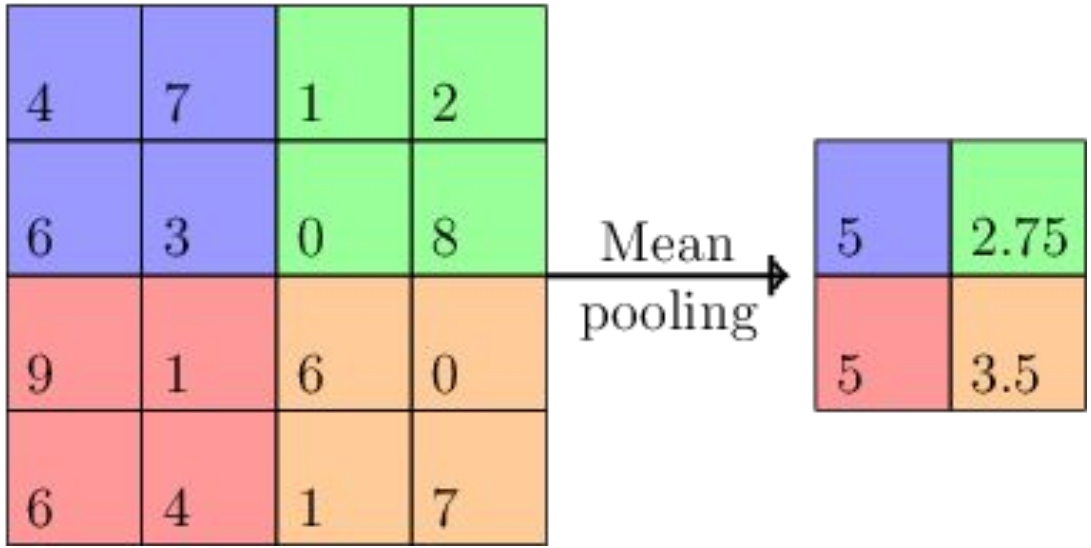
Max pooling:



https://codegolf.stackexchange.com/questions/195348/implement-the-max-pooling-operation-from-convolutional-neural-networks

CNN: Pooling Layer

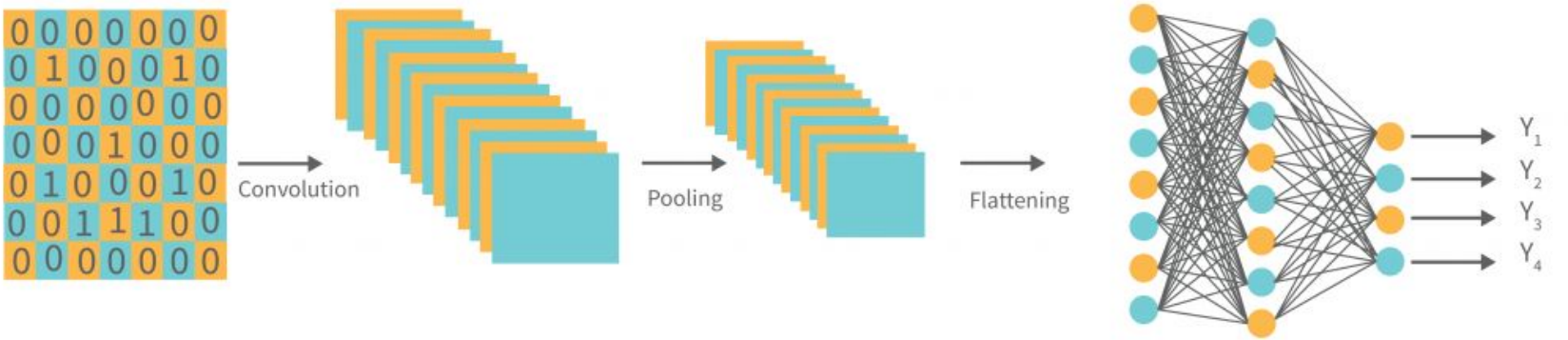
Mean pooling:



https://www.kaggle.com/discussions/questions-and-answers/59502

Full CNN Architecture

After convolution and pooling, we return to normal NN layers:



https://www.interviewbit.com/blog/wp-content/uploads/2022/06/Typical-CNN-Architecture-1024x374.png

Preventing overfitting

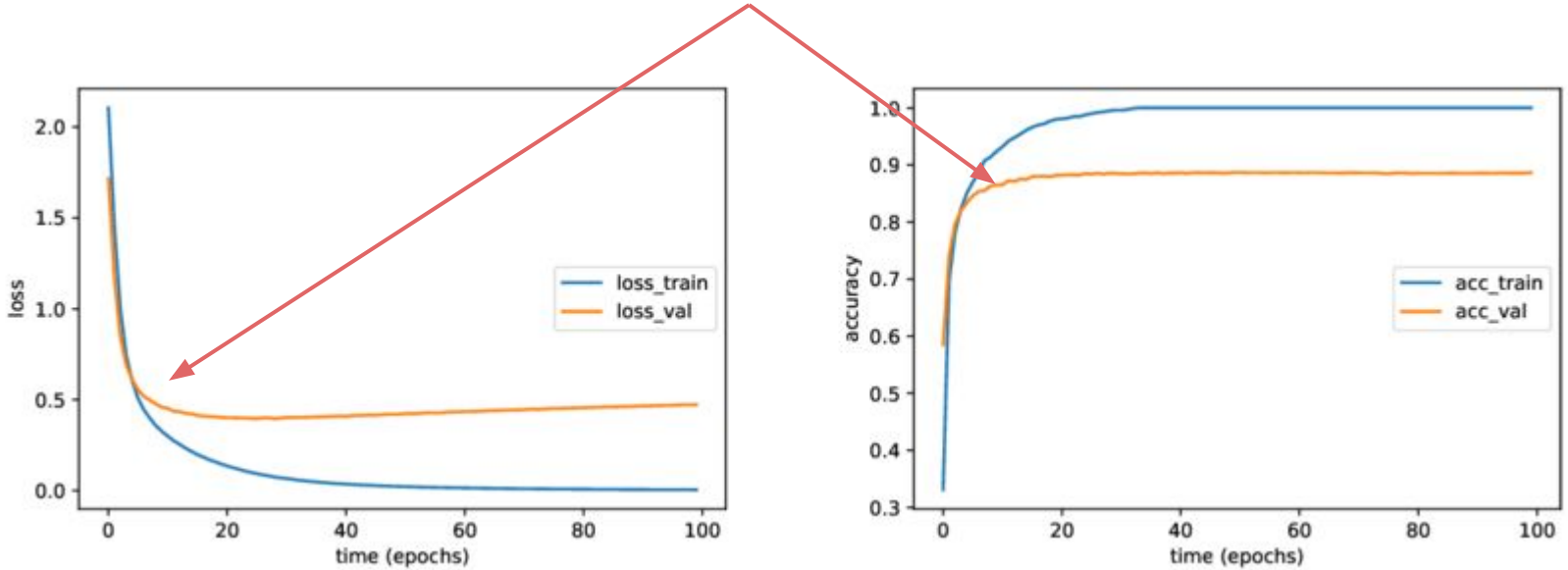
Test, Train and Validation

Problems with having only train dataset

1. When to stop training before the model overfits?
2. How well does the model perform in real life / unseen data?

Split: Train vs Validation

begins memorizing train data



https://www.researchgate.net/figure/Overfitting-of-the-model-on-MNIST-dataset-a-shows-that-after-a-certain-epoch-the_fig2_330553571

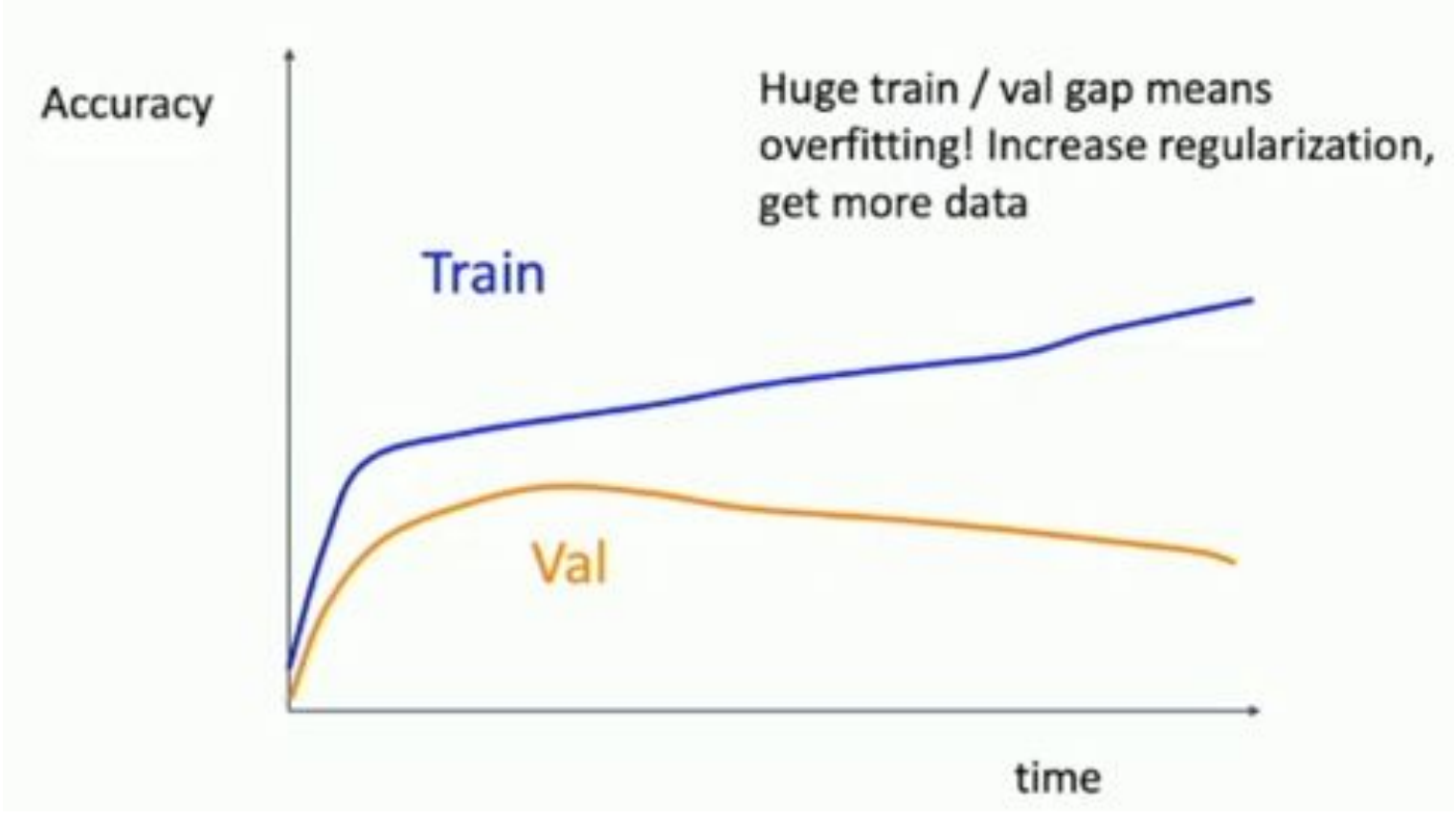
Split: Test Dataset

Final evaluation after training to determine real life performance

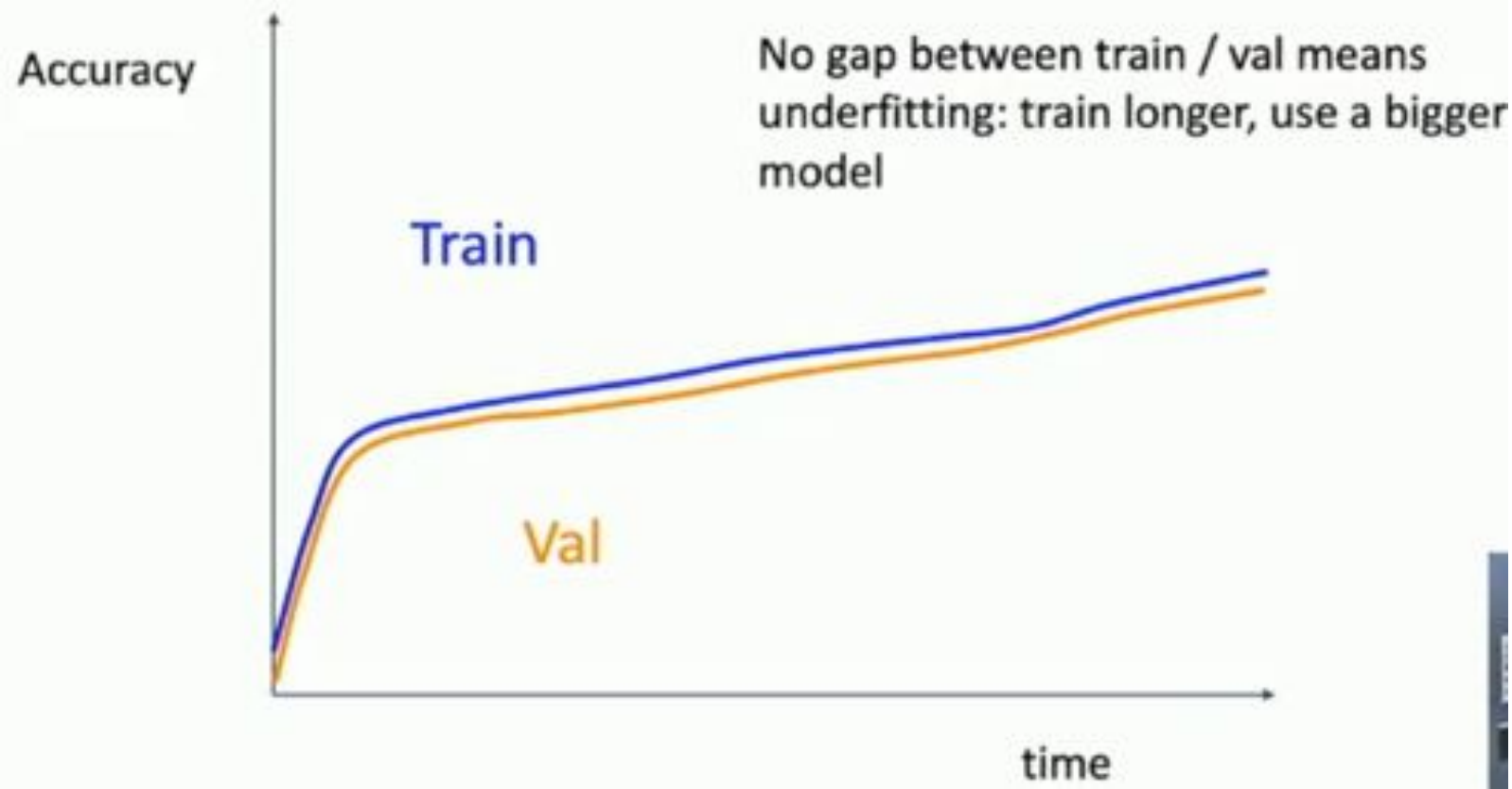
Typical train/validation/test ratios are close to

80/10/10

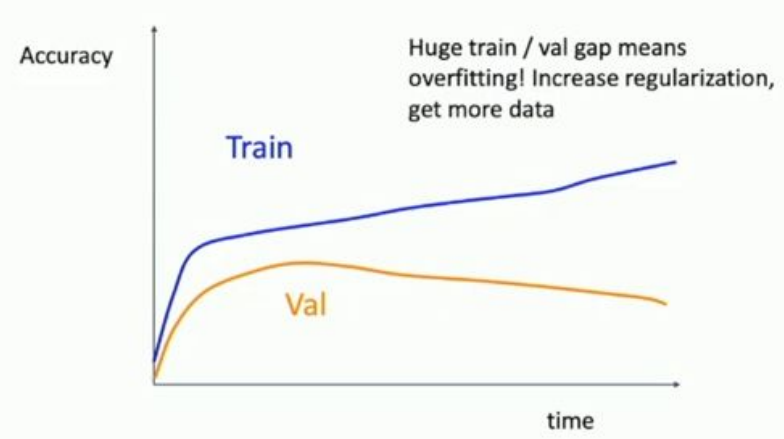
More Graphs: Overfitting

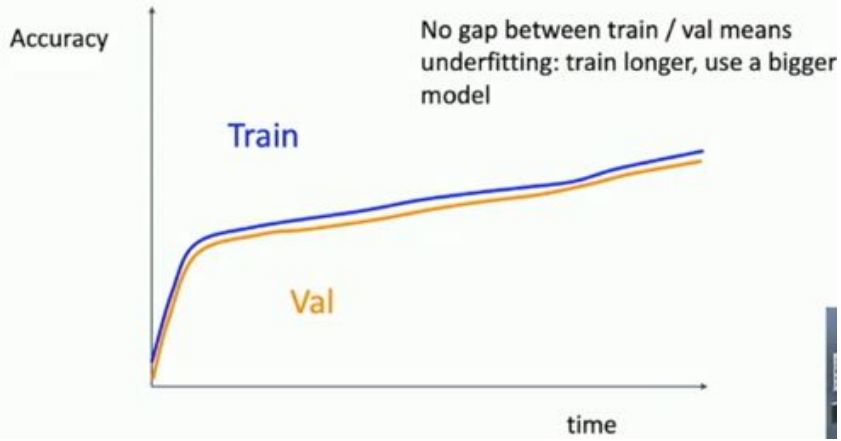


More Graphs: Underfitting



Overfitting vs. Underfitting





Other Hyperparameters

Hyperparameters: Recall

- Learning rate
- Number of layers
- Neurons per layer
- Activation functions
- Kernel size
- Number of Kernels
- Image padding
- Number of iterations

...

New Hyperparameters: Regularisation

Problem: Model is overfitting

New Hyperparameters: Regularisation

Solution 1: Penalize large weights

Prevent heavy reliance on single neurons

L1 Regularization

$$R(w) = \sum_j |w_j|$$

L2 Regularization

$$R(w) = \sum_j w_j^2$$

New Hyperparameters: Regularisation

Solution 1: Penalize large weights

Add penalty to loss function

New loss function

$$L_{new} = L + \lambda \cdot R(w)$$

λ is how much the penalty is weighed

New Hyperparameters: Regularisation

Solution 2: Dropout

During training, randomly set neurons to 0 to some probability p.

When using, disable dropout and multiply all weights by p.

New Hyperparameters: Normalization

Problem: Exploding/Vanishing Gradients due large number of layers → difficult training

Solution: Normalize the dataset to have:

1. Average value of 0
2. Standard deviation of σ

New Hyperparameters: Normalization

Normalization:

$$Z = \frac{X - \bar{X}}{\sqrt{Var(X) + \epsilon}}$$

Mean: $\bar{X}$

$$\bar{X} = \frac{1}{N}\sum_{i=0}^{N} x_i$$

Variance: $Var(X)$

$$Var(X) = \frac{1}{N}\sum_{i=0}^{N} (x_i - \bar{X})^2$$

Epsilon: ϵ

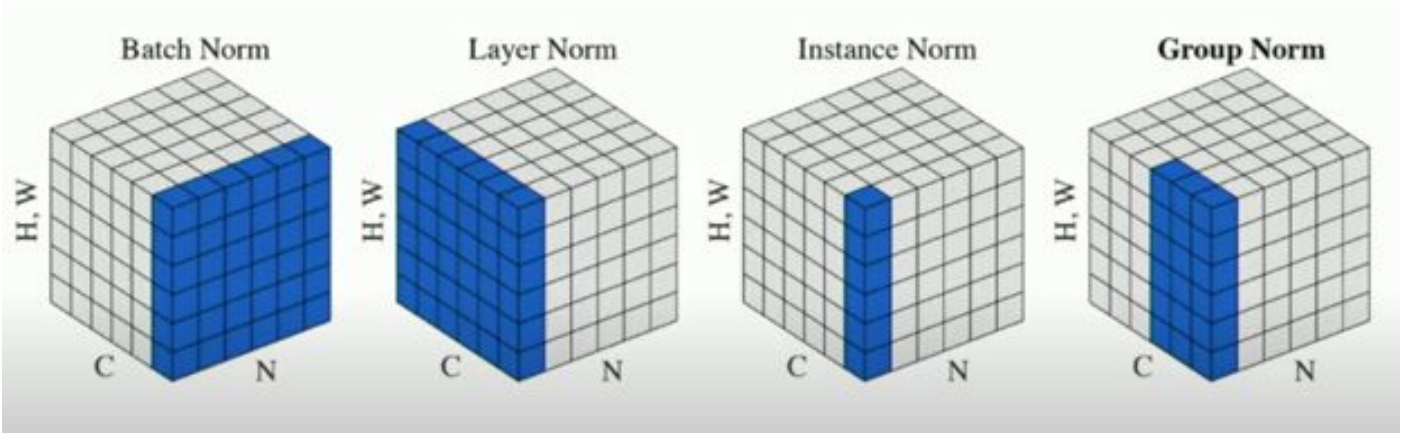
small constant

prevent division by 0

New Hyperparameters: Normalization

Why is this considered a hyperparameter?

Different ways to normalize the data



New Hyperparameters:
Batch size

Recall: we take average loss during gradient descent

How many data points should we process at each iteration?

Usually some power of 2: 32, 64, 128, 256, 512, 1024, 2048

New Hyperparameters:
Batch size

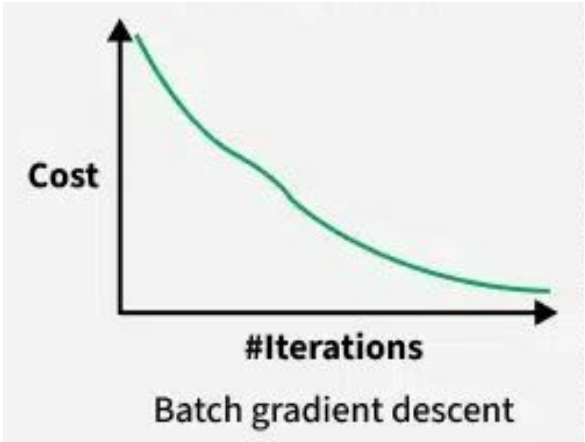
How does batch size affect learning?

Datasets have noise and outliers

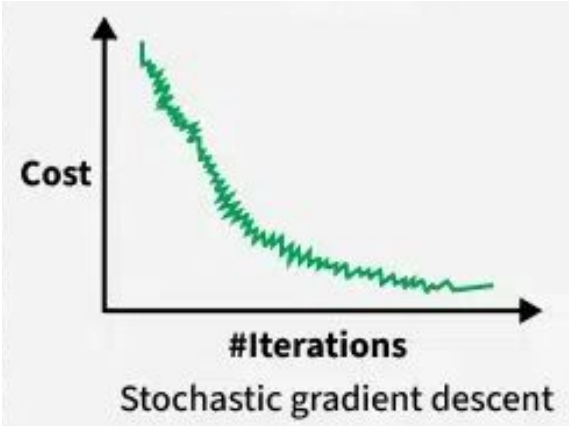
Small batches	Large batches
faster	slower
noisier	more stable

New Hyperparameters:
Batch size

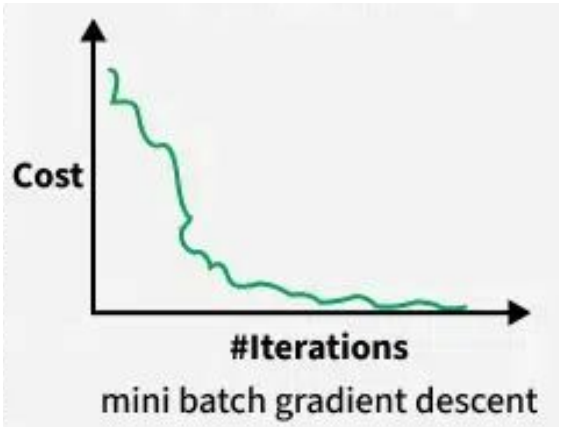
Entire dataset



One datapoint at a time



Small batches



https://www.geeksforgeeks.org/deep-learning/batch-size-in-neural-network/

Optimization algorithm variants

Optimization: Recall

Gradient descent updates model parameters based on the gradient of loss function

Problems:

- gradients can be unstable
- all parameters have same learning rate

Optimization Algorithms Variants

RMSProp

Record running squared average of gradients

to update parameters

Weights running average

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW^2$$

Baises running average

$$v_{db} = \beta v_{db} + (1 - \beta) db^2$$

Optimization Algorithms Variants

RMSProp

Now update parameters based on running average

$$W = W - a\frac{dW}{\sqrt{v_{dW}}+\epsilon} \qquad b = b - a\frac{db}{\sqrt{v_{db}}+\epsilon}$$

Result:

- reduces noise in gradients
- parameters have their own modified learning rates

Optimization Algorithms Variants

Adam

RMSProp +

momentum: normal running averages

Weights running average

$$m_{dW} = \beta m_{dW} + (1 - \beta) dW$$

Baises running average

$$m_{db} = \beta m_{db} + (1 - \beta) db$$

Optimization Algorithms Variants

Adam

RMSProp +

momentum: normal running averages

$$W = W - a\frac{m_{dW}}{\sqrt{v_{dW}}+\epsilon} \qquad b = b - a\frac{m_{db}}{\sqrt{v_{db}}+\epsilon}$$